Comparing Naive Search and KMP

Ethan Goldman

Advanced Algorithms 41052 – Programming Assignment 2 Report

Table of Contents

Abstract *3*

1. Problem and Algorithms *4*

1.1 The Problem 4

1.2 Naive search 4

1.3 KMP 5

1.4 Checking correctness 6

2. Theory and predictions *6*

3. Experimental Methodology *7*

4. Results and Analysis *9*

4.1 Text length and input type 9

4.2 Pattern length and repeated work 11

4.3 Match position and alphabet size 12

5. Surprising result *13*

6. Summary of Findings *15*

7. What to investigate next *15*

8. Individual Contribution *16*

9. AI Use disclosure *16*

9.1 How assistance was used 16

Abstract

This project compares two ways of finding a pattern in text: naive search and Knuth-Morris-Pratt (KMP). Naive search checks each possible starting position. KMP first examines the pattern and uses what it learns to avoid repeating some comparisons. Both algorithms return the first match, or indicate that there is no match.

I compared them on random, repeated and ordinary text, using different pattern lengths and match positions. The results show that neither algorithm is always the quickest. KMP was much faster on a deliberately difficult repeated input, where naive search kept checking long partial matches. On random text and the ordinary-text example, the difference was much smaller. Naive search also did better when the match was right at the beginning, because KMP still had to prepare its pattern table.

A follow-up experiment produced a useful surprise: the algorithms made almost the same number of character comparisons on one input, but KMP still took noticeably longer. This showed that a count of comparisons alone does not describe all the work an algorithm performs. Overall, the comparison illustrates why both worst-case theory and the type of input matter when choosing an algorithm.

1. Problem and Algorithms

1.1 The Problem

The task is to find the first occurrence of a pattern within a text. For example, searching for `ABA` in `CABABA` returns position 1, because counting starts at zero and the first match starts at the second character. If there is no match, the program reports `npos`. An empty pattern is treated as a match at position zero.

The implementation works with bytes. For the examples in this report, it is easiest to think of each byte as a character. In general, though, a character can take more than one byte, so a returned position is a byte position rather than necessarily a position in a line of human-readable text. This distinction matters for things such as accented characters and emoji. The programs do not change capital letters, decode characters or try to find approximate matches.

Both algorithms solve exactly the same problem and use the same rules. In particular, they stop at the first match instead of finding every occurrence. That choice affects the comparison: if a pattern is found at the start, most of the text does not need to be searched at all. A longer text therefore does not always mean more work.

The programs also handle empty inputs, a pattern longer than the text and binary data containing zero bytes. These cases help make the comparison meaningful because a fast answer is only useful if it is correct for the inputs the program claims to support.

1.2 Naive search

Naive search is the straightforward approach. It starts at the first possible position in the text and compares the pattern with the text one byte at a time. If a byte does not match, it moves one position to the right and tries the pattern again from the beginning. It continues until it finds the whole pattern or runs out of positions.

For example, suppose the text is `AABA` and the pattern is `ABA`. Starting at position 0, the first `A` matches but the next comparison fails. The algorithm then starts again at position 1, where it finds the complete pattern. Since it considers positions from left to right, the first complete match it finds is the answer.

The main advantage is simplicity. There is no preparation step or extra table, and the program can return as soon as a match is found. It can also reject a position quickly when the first character does not match. These are useful properties for short searches and inputs with frequent early mismatches.

The weakness appears when many positions almost match. Consider a text made entirely of `a` characters and a pattern made of many `a` characters followed by `b`.

At each position, naive search matches nearly the whole pattern before reaching the `b` and failing. It then shifts by one position and repeats almost the same checks. As the pattern gets longer, this repeated work becomes more expensive.

If the text has length n and the pattern has length m, the worst-case running time is O(nm). This means that, in difficult cases, the amount of work can grow with both the length of the text and the length of the pattern. The algorithm uses O(1) extra space: it needs a few counters but no extra structure that grows with the input. These are worst-case descriptions, not promises about the time taken by every search.

1.3 KMP

KMP takes a different approach. It looks at the pattern before searching the text and builds a table called the longest prefix suffix table, usually shortened to LPS. The table helps KMP work out what can still be useful after a mismatch. The original algorithm was described by Knuth, Morris and Pratt [1].

A prefix is a piece at the beginning of a string; a suffix is a piece at the end. Some patterns repeat parts of themselves, so the end of a matched section can also be the beginning of a new possible match. Instead of throwing that information away, KMP uses the table to decide how much of the pattern is still matched.

For `ABABAC`, the LPS table is `[0, 0, 1, 2, 3, 0]`. The value 3 for `ABABA` means that its ending `ABA` is also its beginning. If a mismatch happens after those five characters have matched, KMP can keep that `ABA` as the start of another possible match. It does not have to pretend that nothing matched.

The example `ABABABAC` searched for `ABABAC` shows why this matters. The first five characters match, but the next text character is `B` while the pattern expects `C`. KMP uses the table to keep the three-character `ABA` match and continues at the same place in the text. It then finds the pattern starting at position 2. If the program simply reset the matched length to zero at that point and continued forward, it could miss this occurrence.

The table therefore serves two purposes. It tells KMP which possible match to try next, and it lets the search keep moving forward through the text instead of moving the text position back to each earlier candidate. KMP may compare a text byte again after a fallback, but it does not restart the search from the beginning of the text.

Building the table takes O(m) time and O(m) extra space. The search takes O(n) time, so a complete one-off search takes O(n + m) time overall. This gives KMP a stronger worst-case bound than naive search. It does not mean KMP always wins in practice: the table must be built even when a simple search would finish almost immediately.

1.4 Checking correctness

Both implementations were tested before comparing their speed. The checks covered empty inputs, a pattern longer than the text, exact matches, absent patterns, matches at the beginning and end, overlapping occurrences, repeated characters and cases where KMP needs to fall back more than once. Binary values, including zero bytes, were also included.

The tests used examples with known answers as well as comparisons with `std::string::find`, a separate standard-library search. Small binary strings were checked exhaustively, and additional cases were generated with fixed seeds so that they could be repeated. The test suite recorded 104,600 answer checks across the search variants, along with checks of comparison counts. Warning-enabled and sanitizer builds also passed.

These tests give good evidence that the implementation behaves as intended on the cases examined. They are not a proof that every possible input has been tested. The explanations of the algorithms are still needed to understand why they should work beyond the test cases.

2. Theory and predictions

Before measuring performance, the main prediction was that KMP would have an advantage on inputs that make naive search repeat long partial matches. The repeated `a` text with a pattern ending in `b` is a clear example. Naive search has to revisit similar sequences at many positions, while KMP's table allows it to keep track of the useful part of the match.

A second prediction was that naive search could be competitive on random inputs. When characters differ early, naive search does very little work at each position. KMP still has its table and fallback logic, so its better worst-case bound may not lead to a faster result for every input.

A third prediction concerned matches at the beginning. Naive search can return after checking the pattern once. KMP must build the LPS table before it performs its search, so that preparation may be a noticeable part of the total time when there is little text to inspect.

These predictions were saved before the main timing experiments and kept after the results were collected. They describe likely behaviour rather than an exact crossover point. There was not enough evidence to predict a precise text or pattern length where

one algorithm would become faster than the other. The experiments were designed to test the different situations rather than assume that a single cutoff exists.

The theoretical comparison gives a useful starting point. Naive search is simple and uses constant extra space, but can take O(nm) time. KMP uses a table that takes O(m) space and gives O(n + m) time for a complete search. In practice, the size of the input, where the search stops and the amount of repeated structure all matter. The experiments show how those differences play out on the selected inputs.

3. Experimental Methodology

The same inputs were given to both algorithms. The experiments included randomly generated text, repeated and periodic text, and an excerpt from Jane Austen's *Pride and Prejudice* [3]. The random examples used different alphabet sizes to change how often partial matches were likely. The repeated examples were included because they can make naive search do the same work many times. The ordinary-text example provided a less artificial comparison, although it was only a small sample of prose.

The tests varied the lengths of the text and pattern. They also included patterns that matched near the beginning, matched near the end, or were absent. Each input was checked to confirm the actual first match. This matters because placing a pattern near the end does not prove that it did not already occur earlier in the text.

The main study used 363 input instances. Each algorithm was run repeatedly on the same instances, and the order of the algorithms was alternated. A short warm-up was used before recorded timings. For very quick searches, several calls were timed together so that the measured period was more useful. The reported values use medians, with the spread between instances shown using the interquartile range.

The timing focused on the search itself. Generating and loading the input, printing results and checking answers were kept outside the timed section. KMP's preparation was included because a one-off search needs to build its table. This is an important part of the comparison and excluding it would answer a different question about reusing a pattern that had already been prepared.

The program also counted character equality comparisons in a separate run. These counts help explain why the timings differ, without adding counting overhead to the main timing. The retained data includes the inputs' settings, run information and raw measurements, and the scripts can regenerate the figures and tables. A small pilot was used to choose a practical timing approach before the main measurements.

All performance measurements were collected on one Apple M1 Pro Mac using an optimized C++ build. The main results therefore describe that machine, those

implementations and the selected input families. The ordinary-text examples came from related sections of one excerpt, not a large collection of independent documents. Linux was used for build and correctness checks, but the performance study was not repeated on Linux.

4. Results and Analysis

4.1 Text length and input type

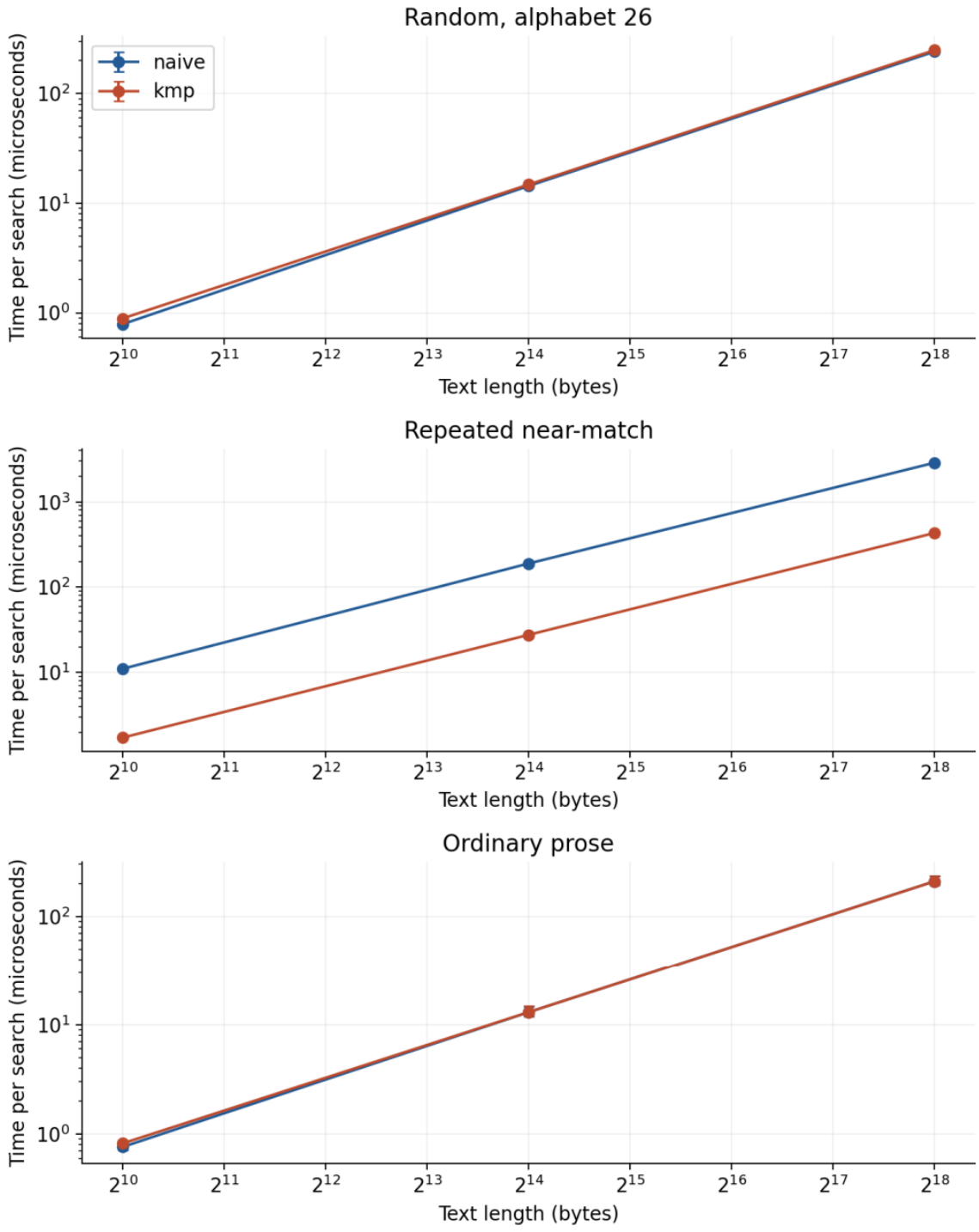


Figure 1: Runtime for absent patterns of length 32 as text length changes. Points show medians, and bars show the spread

When the pattern was absent, both algorithms generally took longer as the text became longer. This is expected: if there is no match, the search cannot stop at a successful occurrence near the beginning. The size of the difference depended on what the text and pattern looked like.

On repeated text with 262,144 bytes and a 32-byte pattern, naive search took a median of 2,878.210 microseconds, compared with 432.520 microseconds for KMP. KMP was about 6.65 times faster in this case. The comparison counts help explain the result: naive made about 8.39 million equality comparisons, while KMP made about 524,000. The repeated near-match made naive check the same sort of prefix again and again.

It would be too broad to say that this graph proves naive search is always much slower as the text gets longer. The pattern length was fixed at 32 for this comparison. With a fixed pattern length, even naive's difficult case can grow roughly in proportion to the text length, although with much more work at each possible position. The next experiment varies pattern length to make the extra repeated work clearer.

The random alphabet-26 example gave a much closer result. For an absent 32-byte pattern in 262,144 bytes, naive took 241.521 microseconds and KMP took 249.271 microseconds. Their comparison counts were also close. Early mismatches help explain why naive did not have to do much repeated work, although the small timing difference should not be treated as a universal result.

The prose example was closer still: the medians were 209.474 microseconds for naive and 209.396 for KMP. The variation between the selected text windows was larger than that tiny difference. This experiment therefore does not establish a meaningful winner for all ordinary text. It shows that the large gap seen on a deliberately difficult repeated pattern was not present in this particular prose case

4.2 Pattern length and repeated work

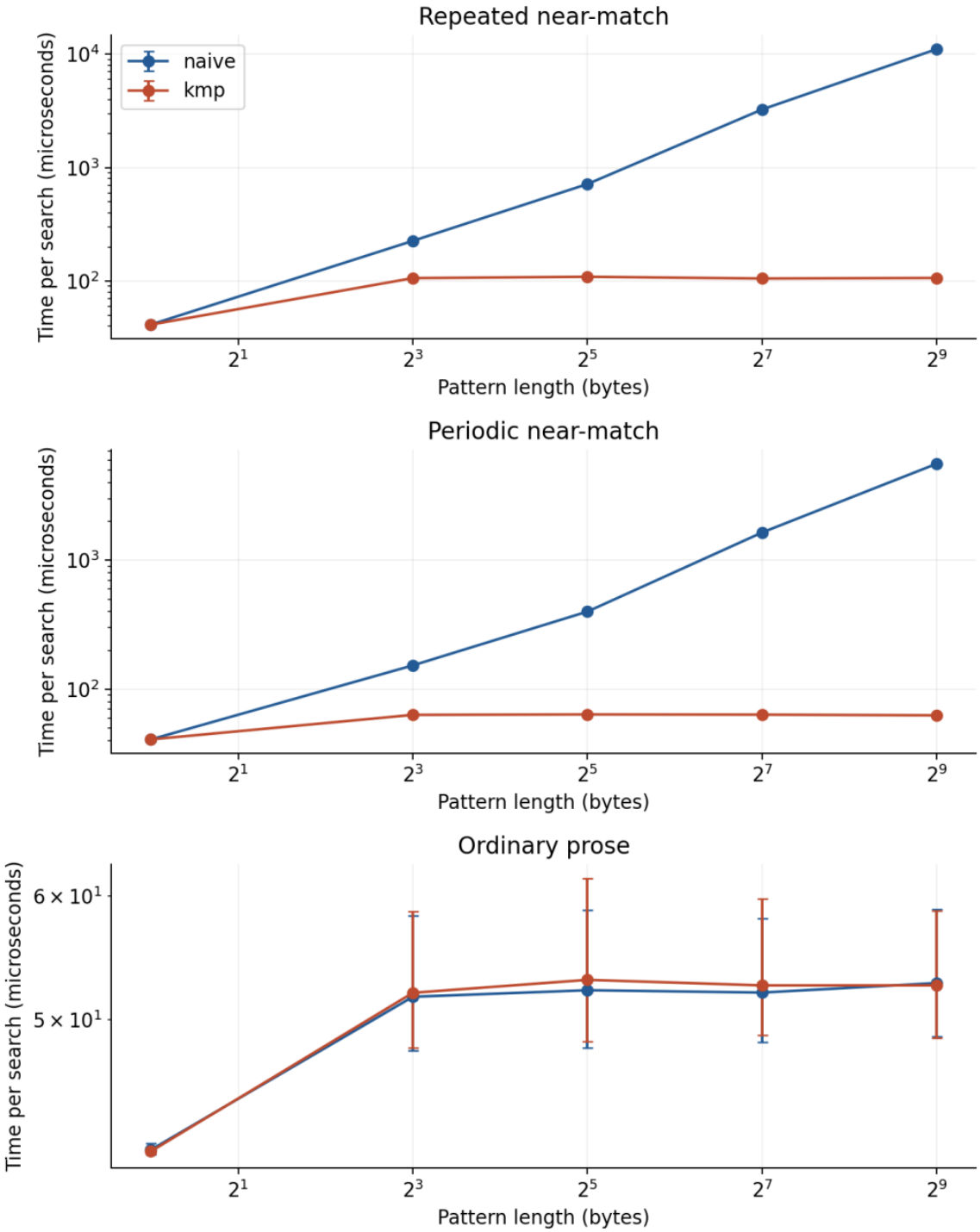


Figure 2: Runtime as the pattern length changes for different input types. The repeated and periodic cases include long partial

The pattern-length results give the clearest example of KMP's advantage. On repeated text of 65,536 bytes with an absent 512-byte pattern, naive search took a median of 11,143.000 microseconds. KMP took 106.656 microseconds, making it about 104.48 times faster on this input. This test was deliberately designed to be difficult for naive search to demonstrate a worst case scenario.

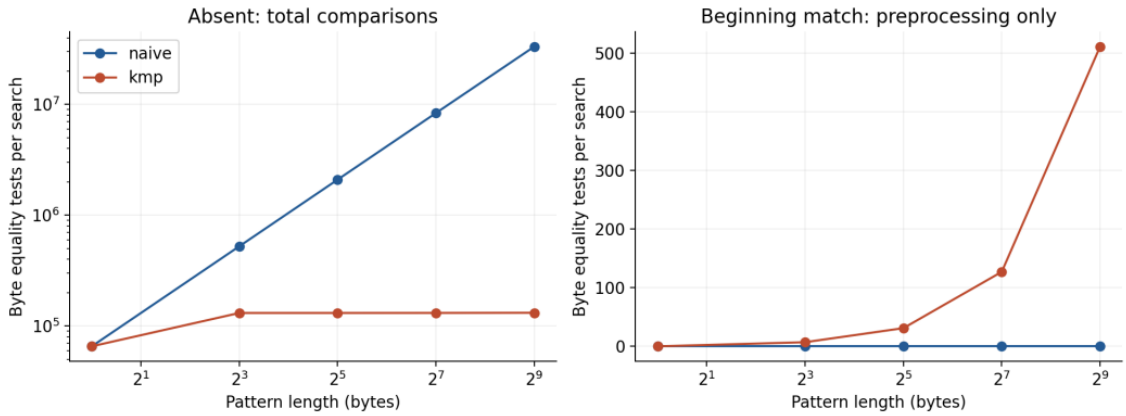


Figure 3: Comparison counts for the repeated-input experiments. The KMP totals include the comparisons used to prepare its

The comparison counts support the explanation. Naive made 33,292,800 equality comparisons in the difficult 512-byte case, while KMP made 131,582 including preprocessing. Naive nearly matched the pattern at each starting point, then failed near its end and began again. KMP used the pattern's repeated structure to avoid much of that repeated checking.

The counts also show a limit of simple measurements. Naive made roughly 253 times as many equality comparisons, but its measured time was about 104 times as long. A comparison is not the only operation either program performs. KMP also updates its current match and consults its table. Comparison counts help show the kind of work being avoided, but their ratio should not be treated as an exact prediction of the timing ratio.

4.3 Match position and alphabet size

A match at the beginning produced a different result. With a 512-byte pattern matching immediately, naive took a median of 0.171 microseconds, while KMP took 0.506 microseconds. Naive could return after checking the pattern. KMP first had to prepare its table and then check the match. This supports the prediction that preparation matters when the search itself is very short.

Patterns near the end or absent usually require more of the text to be examined. That makes the structure of the pattern more important: a long partial match may be cheap when it happens once, but expensive when naive search repeats it at many positions. Checking the actual first match was necessary before making this comparison, especially for random inputs where a supposed late match could occur earlier by chance.

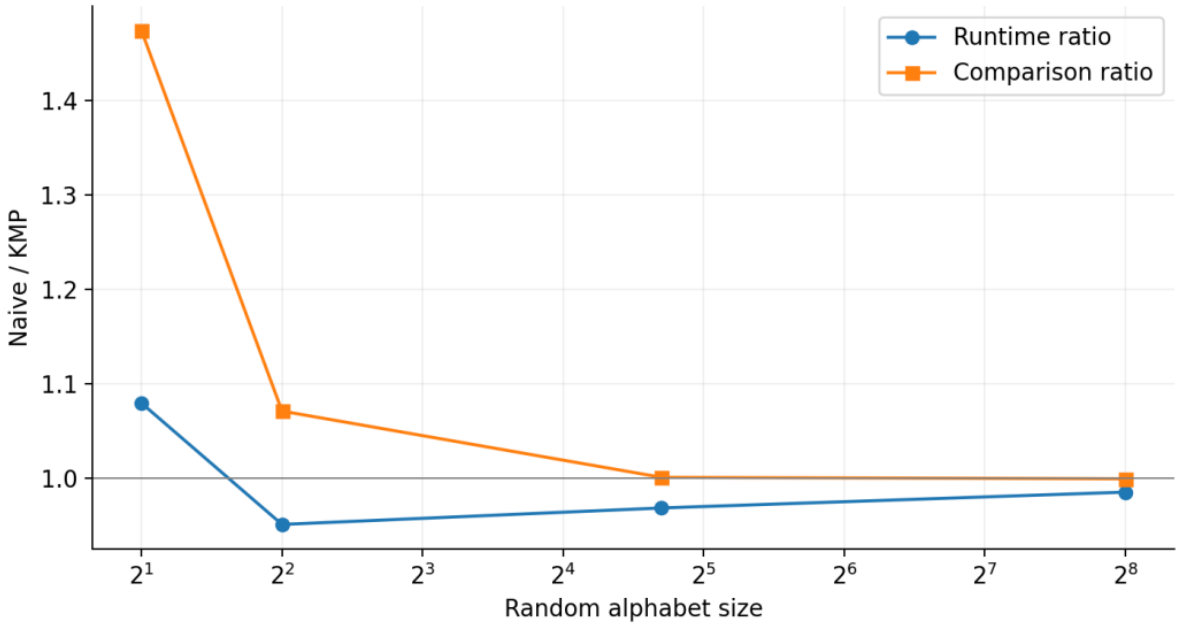


Figure 4: Relative runtime and comparison counts for absent random patterns with different alphabet sizes. A runtime ratio above one means KMP was faster in that case

On the binary alphabet example, KMP made fewer comparisons and was modestly faster: the median times were 1,088.710 microseconds for naive and 1,008.210 for KMP. The comparison-count advantage was larger than the timing advantage. This is another reminder that the algorithms do more than just compare bytes.

The four tested alphabet settings did not produce a simple rule that each smaller alphabet always helped KMP. Alphabet size affects the likelihood of partial matches, but the particular patterns and the work around each comparison also matter.

5. Surprising result

After the main study, a follow-up asked a narrower question: if the two algorithms make almost the same number of equality comparisons, will their runtimes also be similar? The working prediction was that they might be.

The follow-up used a long text of repeated `a` characters and a pattern containing one `b` among otherwise repeated `a` characters. Moving the `b` changed how far the algorithms got before a mismatch. The pattern was absent in each case. This made it possible to examine the effect of mismatch position while keeping the overall structure simple.

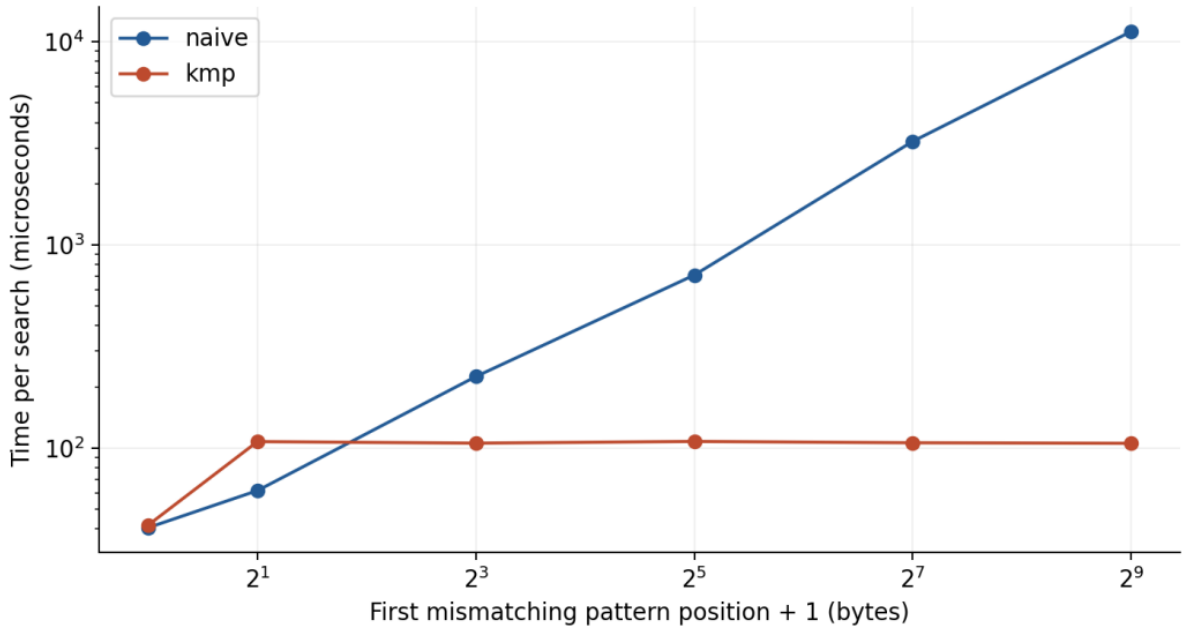


Figure 5: Runtime as the position of the first mismatch changes in the controlled follow-up.

When the `b` was at position 1, naive made 130,050 equality comparisons and KMP made 132,091, including KMP's preprocessing. Those counts differ by only about 1.6 percent. Despite that, naive took a median of 61.691 microseconds and KMP took 107.511 microseconds. KMP was about 1.74 times slower, with a clear separation between the measured spreads for these instances.

The result was surprising because the number of equality comparisons alone suggested a much closer race. Looking at the code helps explain why that expectation was too simple. Naive can reject each candidate after two comparisons. KMP repeatedly falls back in its table and updates its match position before continuing. Both are doing valid work, but the operations surrounding the equality checks differ.

This explanation is supported by the controlled input and the counts, but it has limits. The experiment did not measure cache misses or branch prediction, so it would be speculation to claim either one caused the difference. The supported conclusion is that two algorithms can make similar numbers of comparisons yet still take different amounts of time because those comparisons are part of different sequences of operations.

The follow-up also reinforces the reason for comparing more than one type of input. KMP's preparation and fallback work were a disadvantage in this short-prefix case, while the same overall approach prevented much larger repeated work when the mismatch occurred much later in the pattern. Neither observation cancels the other - they describe different conditions.

6. Summary of Findings

The results show that neither algorithm was faster in every situation. KMP performed much better on repeated inputs that caused long partial matches, because it avoided repeating many of the checks made by naive search. In the most difficult repeated-input case, KMP was about 104 times faster.

Naive search was faster when the pattern matched at the beginning. It could return immediately, while KMP still had to build its table. On the random and ordinary-text examples, the results were much closer.

Overall, KMP provides a stronger worst-case guarantee and is especially useful when repeated patterns could cause a lot of repeated work. Naive search is simpler and can be competitive when mismatches happen early or a match is found quickly.

The comparison also shows why theoretical complexity does not tell the whole story. KMP has a better worst-case time complexity, but its preparation and extra steps still take time. The type of input and where the match occurs affect how much work each algorithm actually does. For the tested inputs, the biggest difference appeared when naive search repeatedly came close to a match before failing.

These findings are based on one computer and a limited set of inputs, so they do not establish one algorithm as the fastest choice in every situation. The results show that the best choice depends on the input and how the search will be used.

7. What to investigate next

One useful next step would be to search many texts using the same pattern. This project measured a complete one-off search, including KMP's table construction every time. If the pattern stays the same, the table could be prepared once and reused. That would be a different workload and should be reported separately, with the cost of preparation and the number of searches made clear.

It would also be useful to test a wider range of real documents and search patterns. The prose study here used related windows from one excerpt and a controlled way of creating absent patterns. More independent texts could show how much the results change between different writing styles and character distributions. Real search queries could add another useful perspective, provided their source and selection were explained.

Finally, the experiments could be repeated on another computer and with another compiler. That would help show which observations are stable and which depend on this machine or implementation. The large difference on the repeated near-match input is supported by a large difference in comparison counts, but small timing differences

deserve more caution. Measurements on other systems would still need correctness checks and a consistent timing method.

The practical lesson from this study is that the choice depends on the workload. KMP is attractive when avoiding repeated worst-case work matters. Naive search remains a reasonable simple approach for small inputs, early matches or inputs that tend to fail quickly. These experiments do not prove or identify a universal length at which one algorithm should always replace the other.

8. Individual Contribution

As I did this assignment individually, everything was done by myself. I implemented both algorithms, testing, benchmarking, report, and presentation individually. Not sure what else to say here ☺.

9. AI Use disclosure

9.1 How assistance was used

I utilised GPT 6.1 Terra exclusively, with a range of reasoning levels through Codex. I utilised AI assistance throughout a large portion of the work I have done, with varying levels of assistance. I primarily wrote the code for both search algorithms myself, and had assistance for other parts, like designing a fair benchmark, analysing results, etc. I also used it to turn my dictated ideas into clearer, more readable writing. Overall, I would rank the contribution as medium-high and would say was used more than the previous programming assignment.

9.2 Examples of problems and corrections

The new Terra model was surprisingly capable, although at times it was almost too proactive. I didn't personally notice any major misinformation like the bitset issue I encountered in the previous assignment. Most of the output I reviewed seemed correct, but the main problem was that the model sometimes did more than I asked, leaving me with less work to do myself.

For example, when I asked for a quick guide on how to design the benchmarking harness, it created the entire harness instead. The result was useful, but building it myself was something I had wanted to do. After seeing a complete implementation, I felt there was less point in writing my own, particularly because I probably would have followed a similar approach.

Another example came when I asked for some statistics from the results to include in the report. The model gave me an explanation that was much more technical and detailed than I needed. I read through the data myself, decided which findings were actually interesting and useful, and added those to the report in a simpler way.

These experiences showed me that assistance can be unhelpful even when its output is mostly correct. Completing too much of the work can reduce my opportunity to practise, and a polished explanation can make it tempting to accept something I don't fully understand. I noticed this behaviour more with Terra than with Sol during the previous assignment, although it seemed to improve towards the end of this project.

9.3 Reflection

Since I completed this assignment individually, I can't reflect on how AI affected group work specifically. I can however individually reflect.

Overall, I can see myself using AI more often, including in day-to-day life. The quality of its responses has improved enough that I sometimes find myself trusting it almost instinctively. That can be helpful when I need a quick explanation or want to turn my ideas into clearer writing, but it also makes me aware that I need to slow down and check what it gives me.

For this assignment, I reviewed the output carefully, especially the code and benchmarking harness. I wanted to make sure both algorithms were tested on the same inputs and that the comparison was fair. I also looked through the results myself to decide which statistics were useful to include in the report, rather than relying on the model to decide what was important.

The biggest risk I noticed was not necessarily that AI would give me an obviously wrong answer, but that it could produce something convincing enough for me to accept without fully understanding it. It also sometimes completed work I had intended to do myself. This made me think more carefully about how much help to ask for and when to step back and work through a problem on my own.

Going forward, I think AI will remain a useful tool for learning, checking ideas and improving my writing. However, I want to be more deliberate about how I use it: asking for explanations or guidance when I want to practise something myself, and reviewing generated work before relying on it.